

流水线并行流式架构的级联式语音对话系统延迟优化

摘要：针对级联式语音对话系统在长语音场景下端点后等待时间随输入时长增长的问题，提出流水线并行的流式优化架构，使语音识别、语言模型推理可重叠执行。前端以语音活动检测实时分段，结合 Whisper 时间戳对齐构建自适应滑窗与动态缓冲，通过前缀-后缀上下文与局部一致性约束提交稳定转录；后端采用键值缓存增量预填充，仅对新增文本更新缓存，降低首 token 输出时间并限制其不随长度线性增长。在由 MultiWOZ（英文）与 CrossWOZ（中文）合成的 1132 条递增长度语音测试集上，长语音分组的平均首 token 输出时间稳定在约 1.1 秒，相比非流式基线降低 34.6%–83.9%，最长分组平均减少 5.67 秒且转录错误率保持在可接受范围。结果表明该架构可有效降低长语音交互等待。

关键词：流式架构；语音对话系统；流水线并行；增量预填充；端到端延迟

文献标志码：A 中图分类号：TP18

Latency Optimization of Cascaded Voice Dialogue Systems with a Pipeline-Parallel Streaming Architecture

Abstract: A pipeline-parallel streaming architecture is proposed to mitigate post-utterance waiting time in cascaded voice dialogue systems for long-form speech. The architecture overlaps speech recognition and language-model prefilling/inference. Voice activity detection enables online segmentation, and an adaptive sliding window with dynamic buffering leverages Whisper timestamp alignment to commit stable transcripts under a prefix-suffix context and a local-consistency constraint. Incremental prefilling with a key-value cache updates only newly arrived text, reducing the time to first token and preventing it from increasing linearly with input length. Experiments on 1,132 progressively-lengthening utterances synthesized from MultiWOZ and CrossWOZ show that mean time to first token remains around 1.1 s in long-utterance groups, achieving 34.6%–83.9% reductions over a non-streaming baseline and a 5.67 s average reduction in the longest group, while keeping transcription error rates within an acceptable range. Results indicate that streaming pipeline parallelism and state incrementalization effectively reduce long-utterance interaction latency in modular cascaded systems.

Key words: streaming architecture; voice dialogue system; pipeline parallelism; incremental prefilling; end-to-end latency

随着人工智能技术的发展，人机交互正由指令式向自然语音对话演进。尽管 GPT-4o、Gemini 1.5 等多模态大模型推动了语音交互能力提升，但受部署成本和工程成熟度限制，当前实际系统仍多采用由自动语音识别（Automatic Speech Recognition, ASR）、大语言模型（Large Language Model, LLM）和语音合成（Text-to-Speech, TTS）组成的级联式架构。在长语音场景中，该架构通常需等待语音端点检测完成后再启动后续理解与生成，容易产生端点后等待延迟，影响交互实时性与流畅性。

相比之下，新一代语音对话系统基于大语言模型有更强的理解能力，能处理复杂推理任务，还能支持带有情感色彩的多轮自然语言交互，可这种智能化水平的提高，也带来了巨大的计算开销和延迟挑战。公开的系统报告指出，GPT-4o 端到端语音响应延迟可低至 232 毫秒，平均 320 毫秒^[1]，这已经接近人类自然对话反应速度。这意味着，低延迟与准确率并列，是下一代语音交互体验的关键竞争指标。

尽管端到端多模态模型在论文和实验中表现出色，可其训练成本高，数据隐私管控难，垂直场景需深度定制。而级联式系统是把语音活动检测（Voice Activity Detection, VAD）^[2]、自动语音识别、

大语言模型和语音合成拆分为独立可替换的模块并按顺序串联，因此可以复用现有的高质量 ASR、文本 LLM 与 TTS 能力。在需要快速迭代 LLM、控制部署成本、降低升级风险的场景中，级联架构仍然具备重要的工程价值。在这种串行架构中，系统的总延迟（ T_{total} ）不仅取决于各个模块的处理时间，还显著受限于模块间的数据流转方式。为方便分析，这种串行级联系统的总延迟可表示为各组件延迟的简单线性叠加：

$$T_{\text{total}} = T_{\text{VAD}} + T_{\text{ASR}} + T_{\text{Prefill}} + T_{\text{Decode}} + T_{\text{TTS}} + T_{\text{Net}} \quad (1)$$

其中， T_{VAD} 表示语音活动检测及端点判定延迟， T_{ASR} 表示语音识别延迟， T_{Prefill} 表示 LLM 对输入上下文进行预填充计算的延迟， T_{Decode} 表示 LLM 自回归生成阶段的解码延迟， T_{TTS} 表示语音合成延迟， T_{Net} 表示网络传输、请求调度及数据序列化等通信开销。需要说明的是，本文的首字延迟（Time to First Token, TTFT）主要用于衡量系统从用户语音输入结束到 LLM 生成首 token 的响应时间，因此统计口径包含 T_{VAD} 、 T_{ASR} 、 T_{Prefill} 以及生成首 token 所需的初始解码延迟，不包含 T_{TTS} 和 T_{Net} 。

在大多数常见的传统级联式实现中，从用户说话结束到系统语音回复往往要等待几秒。特别在长语音场景下，必须等到用户说完整句语音才启动 ASR，而 LLM 的预填充耗时随输入文本长度同步增加。因此，如何在保留级联架构模块化优势的基础上，借助流式并行策略，缩短模块间的“死区时间”，达成打断即响应的交互体验，既有学术意义，又具有工程价值。

1 相关工作

1.1 自动语音识别的流式化进展

ASR 系统的处理速度，决定着用户语音转化为文本 LLM 可接受输入的速度。早期流式自动语音识别的架构演进以 RNN-Transducer（RNN-T）架构为主导^[3]。RNN-T 凭借循环神经网络的天然时序特性，非常适合处理流式音频，但其逐时刻的循环依赖使训练时难以在时间维度上充分并行，并行训练效率受限，且长程语义捕捉较为困难。随着 Transformer 的出现，注意力机制模型的识别精度得以提升到新高^[4]。但是 Transformer 的全局注意力要求必须在整段输入完成后才开始计算，因此原生 Transformer 结构无法支持流式运行。

为了解决这个问题，Google 提出了 Conformer 架构，它把卷积神经网络（Convolutional Neural Network, CNN）的局部特征提取和 Transformer 全局建模结合起来，很快就成了业界主流^[5]。在实际应用中，流式 ASR 常采用分块处理再加上重叠拼接的方式：先用滑动窗口切分音频并识别，然后通过重叠区域对输出做一致性校正，从而在准确率与延迟之间达成平衡。以 Whisper 为例，它的原始模型是采用 Encoder-Decoder 结构的，不过主要面向离线转录^[6]；围绕此类离线模型的流式化，已有工作从模型结构入手，将非因果编码器改造为因果编码器，使其能在更小的分片上实现较低延迟的在线转录，或从训练目标入手，借助序列级正则化激励 token 更早地发射，以缩短流式识别的输出滞后时长。

1.2 大语言模型推理加速技术

在 ASR 完成转录之后，LLM 的推理速度就成了系统延迟的第二大瓶颈。这一过程主要受 Transformer 解码器的自回归生成机制以及 GPU 的显存带宽所限^[7]。键值缓存（Key-Value Cache, KV Cache）技术的出现正是为了解决推理过程里的重复计算问题。该技术通过缓存历史 token 的 Key/Value 向量值，这样在生成新 token 时，就不需要对历史上下文重复计算中间结果了，由此可以将解码阶段的单步计算复杂度从与序列长度二次相关降低为线性相关。现在 KV Cache 已成为 vLLM、TGI

等现代推理引擎的标配^[8]。在长文本输入的预填充（Prefill）阶段，FlashAttention 改进了 GPU 显存的读写方式，采用分块计算策略，让长序列的吞吐量提高到新的高度，从而在一定程度上解决注意力计算量随序列长度迅速增大这一问题^[9]。在长对话场景下，StreamingLLM 引入“注意力汇聚”机制，把早期 token 作为固定锚点，以缓解上下文窗口溢出带来的性能下降，使 LLM 在有限的 KV Cache 里还能持续稳定地流式输出。

1.3 语音对话系统的全链路优化

除了单独优化 ASR 或者 LLM 之外，全链路协同调度对减少整体延迟也很重要。在语音翻译等语音处理任务中，级联方案长期作为常见系统范式存在，其优势在于可复用现有成熟组件、模块替换灵活，但也会带来中间表示损失、误差传播以及模块间延迟累积等问题^[10]。近期端到端流式语音对话模型尝试绕过传统 ASR-LLM-TTS 串行链路，通过语音原生建模和流式生成追求更低的交互延迟^[11]。Mini-Omni、Moshi 和 LLaMA-Omni 等开源系统即属于此类探索。

1.4 本文主要研究内容

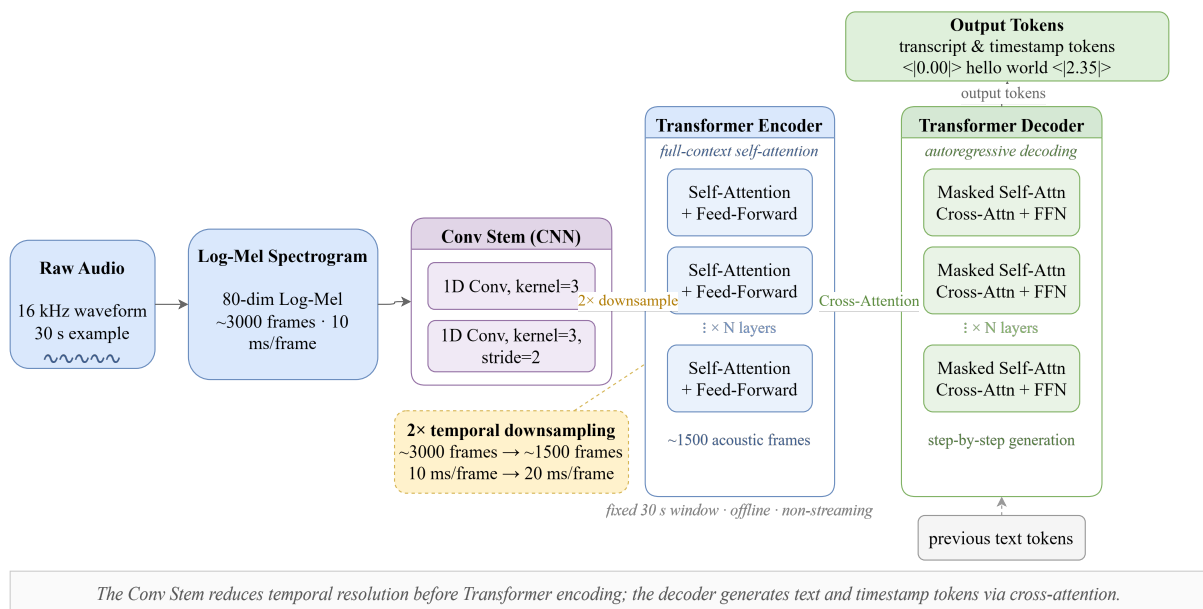


图 1 Whisper 模型整体架构与特征降采样示意图

Fig. 1 Overall architecture of the Whisper model and schematic of feature downsampling

传统级联语音对话系统在处理长语音时，各模块产生的延迟会逐级叠加，同时出现计算资源空转与闲置问题，现有研究要么单独优化 ASR 模型，要么把 LLM 扩展为多模态架构直接生成语音，都难以复用已有的 SOTA 文本模型。本文提出一种细粒度流水线并行方案（下文称 System B），可直接兼容现有非流式 ASR 模型与文本 LLM。本文的主要研究内容与贡献如下：

构建流式 ASR 上下文管理机制：本文设计了基于 Whisper 和 Silero VAD 的自适应滑动窗口算法。为了解决流式切片导致的上下文缺失与识别不稳定性问题，本文提出了动态缓冲与前缀-后缀上下文拼接策略，在保证长句识别准确率的同时，实现毫秒级的 ASR 转录片段输出。

针对长语音输入引发的首字延迟，本文提出了使用 LLM 的 KV Cache 技术，对流式转录产生的文本进行增量预填充。该算法深度运用 Transformer 的 KV Cache 机制，实时接收流式 ASR 的

输出片段，进行注意力计算与状态更新，不重复编码历史上下文，把 LLM 的计算开销均摊到用户语音输入过程中。

本文基于 MultiWOZ（英文）与 CrossWOZ（中文）构建了长语音测试基准，并据此开展全链路端到端延迟评估（测试集的具体构建细节见本文 3.4 节）。实验结果显示，在 15 秒以上的长语音场景里，本文提出的 System B 相比传统非流式基线（下文称 System A）能显著降低延迟，而且没有对系统语义理解能力产生负面影响。

2 原理与方法

2.1 自动语音识别技术

2.1.1 Whisper 模型架构与离线特性

本研究采用 OpenAI 的 Whisper 作为语音转文本模块。Whisper 是基于 Transformer 的 Encoder-Decoder 结构的端到端语音转文本模型，它直接根据语音特征输出文本 token。该模型在大规模弱监督语音-文本数据上预训练，在多语言、口音和噪声场景下具有较强鲁棒性^[12]。

如图 1 所示，Whisper 的输入端将 16 kHz 音频转换为 128 维 Log-Mel 频谱特征（与本文实验所用 Whisper-Turbo，即 large-v3-turbo 一致；早期 Whisper 模型采用 80 维），经卷积前端下采样后送入 Transformer 编码器；解码端通过交叉注意力机制读取编码器表征并自回归输出文本 token。其词表包含时间戳 token，可产生片段级或词级时间对齐信息，这为流式场景中仅提交稳定且可对齐的部分输出提供了实现基础。需要注意，Whisper 默认接口仍偏向固定时长片段的离线处理，这也带来了流式化挑战。

2.1.2 流式化挑战与解决方案

Whisper 的编码器自注意力可利用过去与未来帧信息，离线场景下有助于提升识别准确率，但在实时对话中会带来两类困难：一是短块独立识别时边界附近容易漏词、重复或重写；二是末尾假设会随后续音频到达而改变，下游 LLM 难以稳定消费。

为解决上述问题，本文采用分段输入、重叠上下文、稳定提交的组合策略：前端利用 VAD 对连续 PCM 流进行检测并切分语音段；ASR 侧在目标片段前后保留上下文以缓解边界信息缺失；输出侧利用 Whisper 的词级时间戳，只提交位于稳定区间内的文本片段，从而为 LLM 增量预填充提供可消费的文本流。

2.2 大语言模型推理机制

2.2.1 Transformer 注意力机制

大语言模型通常采用 Decoder-only Transformer 架构。在语音对话系统中，LLM 的推理延迟，尤其是首字延迟，是决定用户是否感到卡顿的关键瓶颈。从推理流程看，TTFT 主要由输入提示文本的预填充计算与随后的首个解码步共同构成，而二者的开销都与输入序列长度密切相关。Transformer 的核心是缩放点积注意力。其标准计算公式如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (2)$$

其中， Q (Query)、 K (Key)、 V (Value) 分别由输入向量经过线性投影矩阵 W_Q 、 W_K 、 W_V 得到。 d_k 为 Key 的维度，用于缩放点积的数值范围，避免 $QK^\top$ 随维度增大而方差膨胀导致 softmax 饱

和、梯度过小。该机制允许模型在一个时间步内显式聚合序列中任意位置的信息，从而捕捉长程依赖关系。

2.2.2 KV Cache 机制的数学原理与复杂度分析

LLM 的文本生成过程本质上是自回归的：模型在生成第 N 个 token 时，需要利用此前 $N - 1$ 个 token 的上下文信息。在自注意力模块中，对长度为 N 的序列执行一次前向计算，其单步复杂度为 $O(N^2)$ 。因此，生成长度为 N 的序列的总计算复杂度为：

$$T_{\text{total}}^{\text{naive}} = \sum_{t=1}^N O(t^2) = O\left(\frac{N(N+1)(2N+1)}{6}\right) \approx O(N^3) \quad (3)$$

尽管现代推理框架通常不会采用完全重复前向的实现，但它直观地说明了自回归生成模式中重复计算的来源：每一步都在重新计算历史 token 对应的 Key/Value 以及它们之间的注意力关系。而 KV Cache 的基本思想就是缓存这些历史值并在后续解码中复用。由于模型参数在推理阶段固定，历史 Token x_i ($i < t$) 的 K_i 、 V_i 一旦计算便不再改变，因此可将其存储在 GPU 显存中。在第 t 步生成时，系统仅需计算当前 Token x_t 的 k_t 和 v_t ，并将其追加到缓存末尾：

$$K_{\text{cache}}^{(t)} = \text{Concat}\left(K_{\text{cache}}^{(t-1)}, k_t\right) \quad (4)$$

$$V_{\text{cache}}^{(t)} = \text{Concat}\left(V_{\text{cache}}^{(t-1)}, v_t\right) \quad (5)$$

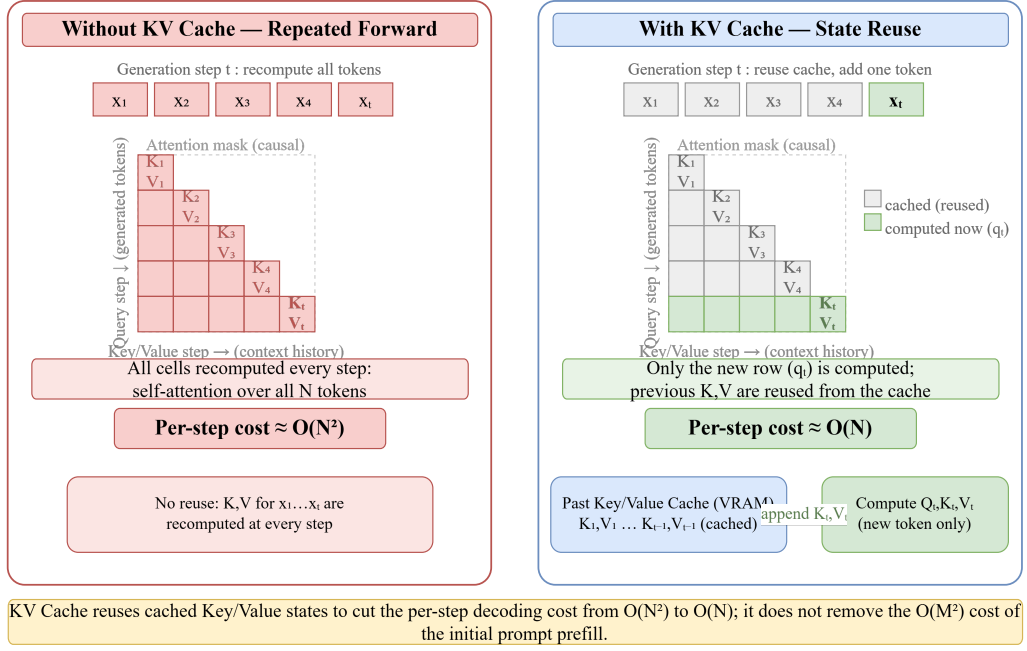
随后，注意力计算仅涉及当前 Query q_t 与历史 K_{cache} 、 V_{cache} 的交互，即计算 $q_t K_{\text{cache}}^\top$ 并加权重 V_{cache} 。因此，KV Cache 将解码阶段的单步复杂度由 $O(t^2)$ 降低为 $O(t)$ ，其仍然会随着上下文长度线性增长，生成 N 个 token 的累计复杂度约为 $\sum_{t=1}^N O(t) = O(N^2)$ ，相应代价是缓存的显存占用随序列长度线性增长。

需要强调的是，本文中 KV Cache 的作用包含两个层次。第一，在 LLM 解码阶段，KV Cache 复用历史 token 已计算得到的 Key/Value，避免生成每个新 token 时反复对完整上下文执行前向计算，从而降低自回归解码的重复计算开销。第二，在本文的流式架构中，ASR 文本并非在语音结束后一次性到达，而是随语音输入逐步产生。因此，系统可以将原本集中发生在语音结束端点 (End-of-Speech) 之后的提示文本预填充拆分为多次增量预填充：每当新的稳定转录片段到达时，仅对新增文本执行前向计算，并将其 Key/Value 追加到缓存中。需要说明的是，增量预填充并不消除 Transformer 对上下文建模所需的注意力计算，也不从理论上改变完整输入处理的复杂度阶数。其主要收益在于计算时序的改变：将长文本提示的大部分预填充计算前移到用户发声过程中，使其与 ASR 和语音输入过程在时间上重叠，从而减少语音端点检测完成后到首 token 生成之间的等待时间，即降低用户感知到的 TTFT。图 2 展示了传统一次性预填充与本文增量预填充与 KV Cache 复用机制在计算路径上的差异。

2.3 系统评价指标

2.3.1 首字延迟

TTFT 是衡量交互实时性的核心指标，直接关联用户的主观等待感。本文将 TTFT 定义为从语音结束端点到 LLM 输出首 token 的时间差，并将其视为端点后关键路径延迟的综合体现。在 System B 中，由于部分预填充计算可在用户发声过程中被增量预填充策略前移并与 ASR 并行处理，因此预计端点后的等待相较 System A 显著降低。



(a) System A: 原生推理 (No Cache) (b) System B: 增量推理 (KV Cache)

图 2 基于 KV Cache 的推理机制示意

Fig. 2 Schematic of the KV Cache-based inference mechanism

2.3.2 词错误率

虽然本研究的核心目标是降低延迟，但这不能以牺牲识别准确率为代价。词错误率（Word Error Rate, WER）是评估 ASR 准确率的通用标准，基于 Levenshtein 编辑距离计算：

$$WER = \frac{S + D + I}{N_{\text{ref}}} \quad (6)$$

其中 S 为替换（Substitution）、 D 为删除（Deletion）、 I 为插入（Insertion）的错误数量， N_{ref} 为参考文本的总词数。通常以百分比形式呈现，此时可再乘以 100%。对于中文数据集，本研究主要考察字符错误率（Character Error Rate, CER），其计算逻辑与 WER 一致。

3 系统设计

3.1 总体架构设计

3.1.1 目标与逻辑拓扑

本系统的核心设计目标是实现从传统的“全量接收-全量处理”范式向“增量接收-流式处理”范式的转移，从而在理论边界上最小化首字延迟。系统的逻辑拓扑如图 3 所示。图中展示了基于多线程生产者-消费者队列的流式并行数据流向。音频以固定时长的 PCM 块持续进入分段模块，Silero VAD 在累计缓冲区上进行活动检测并输出语音段^[13]；ASR 模块将若干语音段拼接后调用 Whisper 进行转录，并按前缀/后缀上下文策略输出稳定文本片段；文本片段经线程安全队列传递给 LLM 模块，持续更新 KV Cache 并在收到终止标记后启动生成，从而尽可能将 LLM 的预填充计算前移并与音频输入过程重叠。

整体架构采用分段-转录-预填充-生成的流水线并行方式：当上游模块产生新的音频或文本段时，下游模块即可立即启动计算，无需等待整段输入结束。系统由以下三个子系统组成：负责语音活动检

测与分段的 Streaming Audio Segmenter、负责上下文拼接并保证输出稳定的 Context-Aware ASR Engine，以及负责增量预填充与最终生成的 Incremental LLM Inference Service。其中 LLM 部分的 KV Cache 属于 Transformer 解码器推理阶段的通用优化思路，本文直接复用推理框架内置的 `past_key_values` 机制完成状态复用。

3.1.2 关键工程实现

为了在单机原型中保持流式链路的实时与可控，工程实现上需要把整条链路拆成多线程流水线，并用线程安全队列解耦，同时借助状态对象保存跨片段的上下文。

在通信机制上，为了去耦合，各模块不再使用同步函数调用，而是使用生产者-消费者模型进行异步通信。在代码实现方面，音频块队列、音频分段队列以及文本段队列都由 `queue.Queue` 的实例承担，消费者端则使用带超时参数的 `get` 方法循环，配合指定的结束事件标记生产端的消息生产结束，以避免单点阻塞，防止整个处理流程卡死。在 ASR 阶段，系统一方面需要承接上传来的分段音频，另一方面又要进行音频转录文本的流式处理，以将流式文本发往下游，因此该系统拆分为收集器与转录器两个子线程：前者持续接收上游语音段并写入等待队列，后者在满足触发条件时批量拼接，再调用 Whisper 进行文本转录，并将转录文本中稳定的文本片段推送给下游 LLM，从而实现了音频收集与模型推理的解耦。

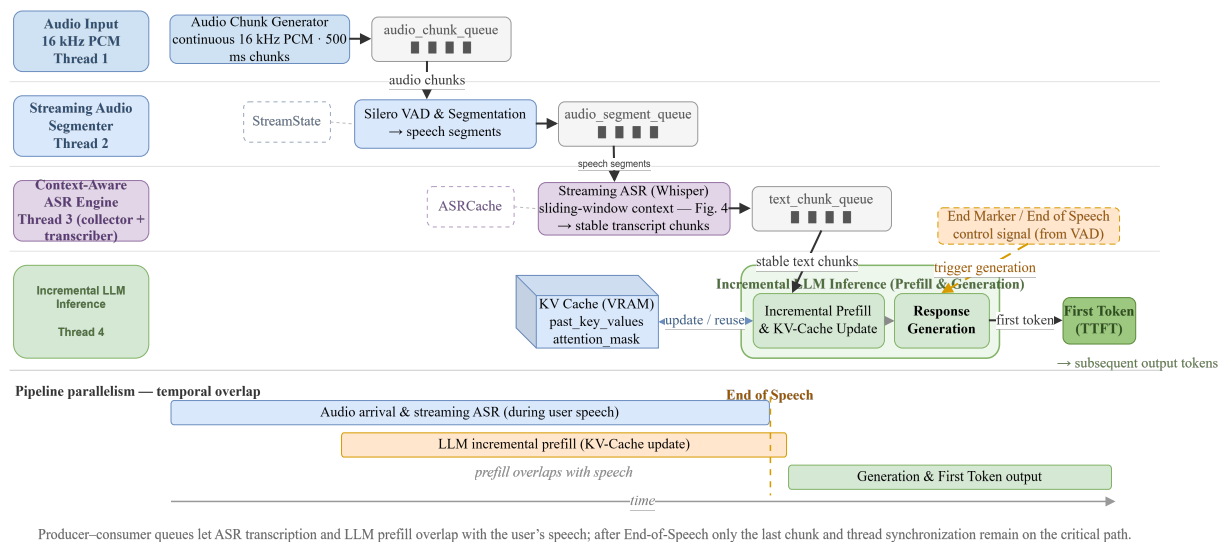


图 3 流式并行架构逻辑拓扑图

Fig. 3 Logical topology of the streaming parallel architecture

而状态管理系统则是采用了有状态与无状态的混合设计。分段模块通过 `StreamState` 维护累计音频缓冲、语音段起止时间与段编号等信息；ASR 侧通过 `ASRCache` 管理等待加入的段队列与当前转录窗口段队列，并使用 `total_duration` 与处理标记避免并发竞争；LLM 侧则以 `KVCache` 持久化 `past_key_values` 与 `attention_mask`、以及预填充阶段产生的最后一步 `logits`，使得终止标记到达后可以直接进入生成阶段。上述状态对象共同保证了跨片段上下文的一致性，并为后续 3.2 与 3.3 的算法描述提供了相应的依据。

3.2 流式 ASR 上下文管理

3.2.1 策略

流式 ASR 的核心挑战在于寻找识别准确率与识别延迟之间的最优平衡：过短的音频切片会导致模型缺乏必要的声学上下文，从而因归纳偏置不足导致字词错误率急剧增加；而等待过长的切片则会引入显著的缓冲与处理延迟，削弱流式处理的优势。为此，本节提出一种基于动态滑动窗口的自适应上下文管理策略。

3.2.2 动态 VAD 分段策略

系统采用 Silero VAD 模型对连续 PCM 音频流进行在线分段。定义输入音频流为时间序列 $\{x_1, x_2, \dots, x_t\}$ ，分段器维护累计缓冲区 A_{accum} ，并在每次接收新的音频块 c_t 后执行 $A_{\text{accum}} \leftarrow A_{\text{accum}} \oplus c_t$ ，其中 $\oplus$ 表示拼接。当累计音频长度超过最小检测窗口后，系统调用 Silero VAD 进行语音活动检测；若检测到足够长的静音，且语音段长度超过最小语音阈值，则认为该语音段已闭合，并输出给下游 ASR。本文原型系统的实验配置为：音频块长度 500 ms、最小语音长度 2 s、最小静音时长 300 ms。上述参数主要基于实时性与分段稳定性的折中选择。500 ms 块长能够提供较细的在线检测粒度，同时避免过短块长带来的频繁 VAD 调用和线程调度开销；2 s 最小语音长度用于保证每次提交给 Whisper 的语音片段具有足够上下文，减少过短片段导致的识别不稳定；300 ms 静音阈值则用于区分句内短暂停顿与真实语音结束，避免系统过早截断语音段。从参数敏感性看，较小的音频块长或静音阈值有助于降低端点检测等待时间，但可能增加误分段和边界处识别抖动；较大的块长、最小语音长度或静音阈值则可提升分段稳定性，但会增加端点后等待延迟。因此，本文在所有对比实验中固定上述 VAD 参数，以保证不同系统配置之间的公平比较。

3.2.3 基于滑动窗口的上下文感知

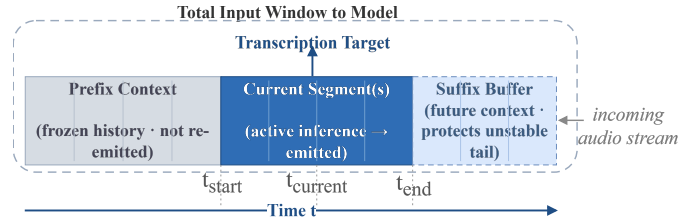


图 4 上下文感知 ASR 滑动窗口机制示意图

Fig. 4 Schematic of the context-aware ASR sliding-window mechanism

为了解决流式识别中的边界效应，ASR 引擎引入了包含前缀上下文的滑动窗口机制。图 4 详细展示了时间步 t 时 ASR 模型的输入构成。窗口被划分为三个逻辑区域：前缀上下文为历史已确定的冻结区域，提供必要的声学历史信息；当前片段为待识别的目标推理区域；后缀缓冲则包含未来音频信息以防止截断效应。系统通过这种重叠切片策略，确保了流式识别结果的连贯性。

首先，执行窗口更新。设 Q_t 为时刻 t 的音频段队列。当新的音频段到达时，系统先将其加入等待队列，并在转录线程中批量并入主队列得到 Q_t ；当队列累计时长满足触发阈值 τ_{asr} ，且队列长度满足要求，具体如公式 (7) 所示。

$$|Q_{\text{temp}}| \geq N_{\text{prefix}} + 1 + N_{\text{suffix}} \quad (7)$$

其中 $|Q_{\text{temp}}|$ 为当前临时队列长度， N_{prefix} 与 N_{suffix} 分别表示系统配置需要的前置队列与后续队列片段的长度。当满足条件或者收到了最后一个语音段时，ASR 模型 M 对临时队列中的所有音

频段进行拼接后执行一次文本转录，生成原始转录结果。在工程实现中，往往还会增加一个最短识别阈值进行控制，用于避免过短窗口造成的频繁调用与不稳定输出。

其次，进行确定性文本提取。鉴于 Whisper 等 ASR 模型在处理流式输入的末尾时往往存在“闪烁”现象：即随着新音频的输入，原本输出的末尾词汇可能发生变化，因此系统在输出侧加入“后缀保护”段以增强输出文本的稳定性，系统仅输出前缀段与后缀段之间的稳定区域内的文本。为保证输出与音频段一一对应，本文实现利用 Whisper 自带的词级时间戳将原始转录结果中的词按结束时间映射回每个音频段的时间区间，从而得到段级候选文本；随后仅拼接稳定区域内段的文本形成本轮输出。在工程实现中，系统还对流式起止边界做了特殊处理：在流式起始轮时，前缀段尚未输出过，稳定区的起点可从 0 开始；在流式结束轮时，为避免遗留文本，系统会输出所有剩余段并完成收敛。最后，执行缓冲区状态迁移。在输出稳定区域文本后，窗口执行滑动以限制累计长度并维持声学连贯性：系统不会将队列全部清空，而是保留最后一个已输出段之前的 N_{prefix} 个段作为下一轮的前缀上下文，同时继续保留尚未输出的后缀保护段。需要指出的是，稳定输出的代价并非仅增加一个分片的固定延迟，而是至少需要等待 N_{suffix} 个后续段到达、并满足触发阈值 τ_{asr} 。但该代价换来了输出文本在边界处的稳定性与准确性，使下游 LLM 能够稳定地进行增量预填充，从系统层面实现端到端延迟可控下降，同时保证输入的准确性。

3.3 LLM KV Cache 增量预填充策略

3.3.1 策略说明

在 System A 中，LLM 需等待 ASR 完成整段语音转录后才能获得完整提示文本，并在语音结束端点之后集中执行预填充与首 token 的生成。因此，随着转录文本长度增加，TTFT 通常呈显著上升趋势。若将该串行范式直接应用于音频输入期间的增量计算场景，则无法有效利用语音结束端点之前的时间窗口，导致潜在的并行计算机会被浪费。

为减少语音端点后的计算量，支撑流水线并行，System B 在 LLM 部分引入了增量预填充：语音端点之前，使用已经产生的流式文本进行预处理，在 Transformer 解码器预填充时产生的 `past_key_values` 作为 KV Cache 的历史，保存在内存或 GPU 显存中，在新文本片段抵达时，利用保存的 `past_key_values`，只对新增 token 做前向计算，再将计算后新的 `past_key_values` 替换旧值，在另一段新文本到达时重复上述过程。这样 LLM 的大部分增量预填充可以在用户说话期间完成。当终止标记到达时，系统只需处理末尾片段，马上开始生成回复 token 的计算工作，从而减少语音结束到首 token 输出的等待时间。

3.3.2 核心算法实现

增量推理的核心逻辑：首次调用时，系统在无缓存状态下一致性执行增量预填充，建立初始 KV Cache；随后，每当上游传来一段新的 ASR 文本片段，系统便触发一次调用 `cache_prompt()`：在已有 `past_key_values` 的基础上做增量更新。当收到终止标记时，系统在提示文本末尾追加生成提示符（例如“assistant:”，用于告诉模型后续输出的首 token 即是智能助理回复），并复用预填充阶段最后一步的 logits 直接开始解码。为避免增量 token 的位置编码错位，实现里会显式构造 `position_ids`，同时将新增片段以不增加生成提示符的方式编码，确保缓存拼接可控。Algorithm 1 给出了与该实现对应的增量更新过程。图 5 直观展示了显存内的状态变化。左侧蓝色区域表示历史上下文对应的缓存键值对，右侧绿色区域表示新增片段产生的键值对。增量预填充在新片段到达时复用历史 `past_key_values`，仅对新增 token 执行前向并更新缓存，从而避免在增量场景下反复对全量提示文本进行预填充。

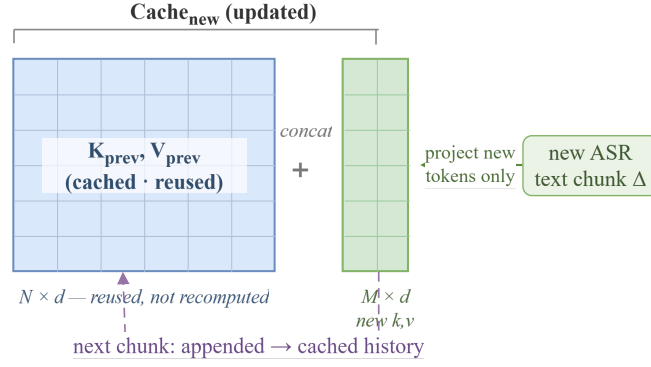


图 5 LLM KV Cache 增量更新机制示意图

Fig. 5 Schematic of the LLM KV Cache incremental update mechanism

3.3.3 复杂度分析

为了说明增量预填充减少冗余计算的来源，考虑历史上下文长度为 N 、新增片段长度为 M 的一次更新：若在每次增量到达时都对全序列 $x_{1:N+M}$ 重新执行一次预填充（不复用历史状态），则自注意力的主要开销近似为 $O((N+M)^2)$ 。而在使用 KV Cache 的情况下，历史 token 的 K, V 已经缓存，系统仅需对新增 token 计算 Query 并与长度为 $N+M$ 的 Key/Value 交互，主要开销可写为 $O(M(N+M))$ 。当 $M \ll N$ 时，上式近似为 $O(MN)$ ，体现为对新增片段的线性扩展。需要强调的是，若将所有增量片段累积到最终长度 L ，增量预填充的累计开销为

$$\sum_j O(M_j L_j) = O(L^2) \quad (8)$$

其中 M_j 与 L_j 分别为第 j 个新增文本序列的长度与累积长度， L 为完整序列长度。与离线一次性预填充同阶。本文策略的关键收益不在于改变复杂度阶数，而在于将预填充计算前移到用户发声期间，使语音结束端点之后需要完成的计算主要集中在最后一小段增量及线程同步开销，从而有效降低 TTFT。与此同时，本文实现会在 KV Cache 中保留预填充阶段最后一步 **logits**，使得终止标记到达后无需额外前向计算即可解码首 token，进一步减少首字阶段的常数开销。

3.4 实验数据构建与处理管线

3.4.1 简述

为了在受控环境下系统性验证流式架构的性能，需要有不递增长度的对话数据，本研究构建了一套自动化的数据合成管线。该管线将 MultiWOZ（英文）^[14] 和 CrossWOZ（中文）^[15] 这两个标准多轮对话数据集转化为具有精确时长标注的逐渐递增的长语音测试数据，再通过 TTS 生成音频数据。

3.4.2 累积对话生成逻辑

MultiWOZ 和 CrossWOZ 均包含多轮对话信息，但其中单次用户输入通常较短，难以直接覆盖长语音输入场景。为模拟用户在实际语音交互中一次性陈述复杂需求或连续补充信息的情况，本实验采用累积对话策略构造长输入样本。如表 1 所示，算法沿对话历史逐轮遍历，将当前轮及此前若干轮的用户侧输入与系统回复文本进行累积拼接，从而生成长度递增的输入序列。

通过此策略，能够基于真实的语义语境，生成从 3 秒至 60 秒以上不等的连续语音样本，有效覆盖了从短指令到长篇陈述的各种交互形态，解决了传统数据集缺乏长语音样本的问题。

表 1 对话数据累积方式
Table 1 Dialogue Data Accumulation Strategy

轮次	用户	客服	数据文本
1	订一张机票	去哪里?	订一张机票
2	去北京, 明天出发	要哪种舱位呢?	订一张机票, 去哪里?, 去北京, 明天出发
3	要商务舱	好的, 这就为您预订	订一张机票, 去哪里?, 去北京, 明天出发。 要哪种舱位呢? 要商务舱

3.4.3 数据处理流水线

数据构建过程包含三个严格顺序执行的阶段:

首先是历史累积与筛选。策略遍历源数据集, 应用上述累积生成逻辑。为了聚焦长语音场景下的性能瓶颈, 系统按文本长度对生成样本进行倒序排列, 优先选取最长的前 N 个对话片段, 并设置最大文本长度阈值 (英文 2050 字符, 中文 720 字符) 以防止显存溢出, 且设置语音长度上限可以防止无意义地增加实验时间。

其次是并发 TTS 音频合成。近年来神经语音合成的自然度已接近真人水平^[16], 本研究集成了阿里巴巴提出的 CosyVoice 大模型语音合成服务^[17]。相较于传统 TTS, CosyVoice 能够生成韵律更自然、情感更丰富的高保真语音波形, 更贴近真实人声输入。在工程实现上, 开发了批量处理模块, 采用多线程异步请求机制, 显著提升了大规模数据生成的效率。

最后是时长校准与元数据同步。由于生成式 TTS 模型的语速具有非确定性, 简单的基于文本字数的时长估算往往存在误差。本管线通过解析生成的 WAV 文件头来获取以秒为单位 (可精确到毫秒级) 的物理时长。该真实时长被回写至测试元数据 JSON 中, 作为后续实验中 X 轴 (输入时长) 的真值依据。

3.4.4 测试集分组定义

表 2 音频时长分组
Table 2 Audio Duration Groups

组名	时长 (s)	典型场景	实验聚焦
Short	$T < 5$	短指令	基准延迟验证 (Baseline Latency)
Medium	$5 \leq T < 15$	包含多轮意图的陈述	日常对话性能
Long	$15 \leq T < 30$	复杂长难句/长段落	流式架构核心优势区间
Very Long	$30 \leq T < 60$	长段论述说明	判断 TTFT 是否存在稳定上界
Extra Long	$T \geq 60$	极限压力测试	系统稳定性与显存边界 (Stability & OOM)

为了细粒度分析不同时长下的延迟表现, 将生成的样本集依据音频时长划分为五个标准实验组 (见表 2)。

这一分组标准将在第四章的实验分析中贯穿始终, 用于对比不同系统架构在处理长短语音时的性能差异。

4 实验与结果分析

4.1 实验说明

本章围绕 VAD 判定用户语音结束至大语言模型产生首 token 的首字延迟开展实验验证，系统性评估本文提出的流式级联式语音对话架构 System B 在长语音交互场景中的性能收益与代价。与传统非流式基线 System A 相比，System B 的关键差异在于：其一，将 ASR 从端点后全量解码改造为分片流式解码，使声学计算与用户发声时长发生重叠；其二，在 ASR 增量输出的推动下，对 LLM 进行增量预计算，以削弱长上下文一次预填充带来的串行等待。通过控制变量，本章依次从延迟特性、关键模块消融以及精度-延迟权衡三方面给出实证结果，并讨论观察到的影响延迟的“主导项切换”现象及其工程边界。

4.2 实验一：延迟与语音长度的关系验证

4.2.1 实验设置与环境构建

为了客观衡量系统在长语音场景下的实时响应能力，本实验选取 MultiWOZ 与 CrossWOZ 两个公开数据集，分别代表英文与中文的多轮对话任务^[18, 19]。按上一章节所述的数据构建流程，生成了覆盖短 (Short)、中 (Medium)、长 (Long)、很长 (Very Long)、超长 (Extra Long) 五组长度的混合测试集，共 1132 条样本。其中 MultiWOZ 共 630 条 (短 31、中 41、长 64、很长 112、超长 382)，CrossWOZ 共 502 条 (短 4、中 48、长 57、很长 96、超长 297)；另有 1 条超长组样本因运行故障被剔除，未计入统计。各分组样本数量分布另见表 3。

实验硬件平台统一基于两块 NVIDIA RTX 3090 (24 GB VRAM) 显卡，ASR 模型与 LLM 模型分别部署在两块显卡上运行。为消除模型加载、CUDA Context 初始化及核心算子首次调用带来的随机误差，对比组 System A 与实验组 System B 采用相同的模型权重与运行环境：ASR 端均使用 Whisper-Turbo，LLM 端均使用 Qwen2-7B-Instruct，从而排除模型规模和硬件配置差异对实验结果的影响^[20]。实验记录前，系统使用真实音频执行 3 轮预热，以确保模型权重已加载至 GPU，CUDA kernel 初始化完成，推理速度进入相对稳定状态。在推理配置方面，LLM 采用 Hugging Face Transformers 的 AutoModelForCausalLM 加载，推理时设置为 eval 模式，并使用 torch_dtype="auto"，即采用模型权重配置对应的默认精度；实验代码未额外启用量化推理。所有样本均采用单样本推理，LLM 输入 batch size 为 1；最大生成长度设置为 50 个 token，但 TTFT 仅统计从语音输入结束到首 token 产生的时间。解码参数保持一致，温度系数为 0.1，top_p 为 0.9，重复惩罚系数为 1.1。系统未使用 vLLM 等独立推理框架，也未在代码中显式启用 FlashAttention，而是采用 Transformers/PyTorch 的默认注意力实现。需要说明的是，System A 是常规非流式串行级联实现：完整音频输入结束后进行一次性 ASR 转录，再将完整文本送入 LLM 完成一次性预填充与首 token 解码，使用默认设置时在连续解码阶段也同样会使用 KV Cache，但实验只考察首 token 生成的时间。System B 与 System A 的差异仅在于流式分段、流水线调度和 KV Cache 增量更新方式，而非模型权重、推理精度、batch size、最大输出长度或硬件条件。

4.2.2 延迟趋势分析

首字延迟是衡量语音交互流畅度的核心指标。图 6 展示了在不同语音时长输入下，非流式基线系统与本文流式系统的 TTFT 变化轨迹。

其中 X 轴为音频真实时长 (秒)，Y 轴为端到端 TTFT (毫秒)。System A 曲线随时长近似单调上升，反映端点后串行等待的累积；System B 在长语音组及以上区间趋于平缓，呈现由固定分片

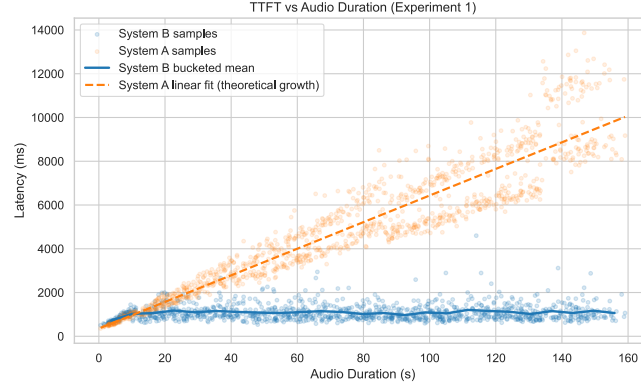


图 6 System A 与 System B 的 TTFT 随时长变化趋势

Fig. 6 TTFT trends of System A and System B over input duration

粒度与尾部调度开销主导的近似上界。两条曲线在 15 秒附近出现拐点，意味着流式并行的覆盖收益开始超过其固定开销。

表 3 不同语音时长分组下的 TTFT 统计

Table 3 TTFT Statistics Across Speech Duration Groups

组名	样本数	平均时长 (s)	流式 TTFT(ms)	非流式 TTFT(ms)	改进 (%)
Short	35	3.44	648.17	533.42	-21.5
Medium	89	9.25	959.53	923.42	-3.9
Long	121	22.16	1126.63	1722.04	34.6
Very Long	208	45.21	1099.16	3191.97	65.6
Extra Long	679	105.73	1087.70	6753.43	83.9

表 3 清晰地给出了按语音时长分组的 TTFT 统计结果。非流式基线 System A 的 TTFT 随输入时长有明显增长。其中 Short 组均值为 533.42ms，Long 组上升到了 1722.04ms，Very Long 与 Extra Long 组进一步增长到 3191.97ms 与 6753.43ms。这一现象和级联式处理范式的预期是相符的，因为 ASR 需等待语音生成完成之后才能进行全量解码，之后 LLM 才会对完整转录文本一次性进行全量预填充。在此期间，端点之后的计算链路几乎不会重叠，这就使得语音长度与 TTFT 基本是线性相关的关系。而流式系统 System B 在 Long 及以上区间显示出近似常数的上界：Long、Very Long 与 Extra Long 三组的 TTFT 均值分别为 1126.63ms、1099.16ms 与 1087.70ms，整体稳定在 1.1s 左右。相应的 TTFT 在 Long、Very Long、Extra Long 这三个分组上，相对于基线系统分别压缩了 34.6%、65.6% 与 83.9%，其中 Extra Long 组平均绝对延迟降低了 5.67s，明显减轻了长语音交互时的静默等待。需要说明的是，这里的常数是主导项由随时长累积的全量计算耗时，转变为最后若干分片的尾部处理与调度开销之和。在音频包时长 500 ms 与计算资源不变的前提下，TTFT 不再随输入时长的增长而继续显著增加。

值得注意的是，在 Short 与 Medium 组中，System B 相较于 System A 分别出现了 114.75 ms 和 36.11 ms 的延迟增加。本文将延迟变化量定义为 $\Delta\text{TTFT} = \text{TTFT}_{\text{SystemB}} - \text{TTFT}_{\text{SystemA}}$ ，因此 $\Delta\text{TTFT} > 0$ 表示流式方案的 TTFT 高于非流式基线，即存在负向收益； $\Delta\text{TTFT} < 0$ 则表示延迟降低。在短语音场景下，分片处理、缓存维护和增量提示构造等流式架构的固定开销难以被并行计算收益抵消，同时较短的语音和文本输入尚未充分触发长上下文预填充的计算瓶颈。该结果表明，

在工程落地时可根据输入时长进行自适应路径选择：对预测时长较短的输入优先采用传统一次性处理链路，而将流式并行机制主要用于音频时长 $T \geq 15\text{s}$ 的长语音场景，以获得更优的整体交互体验。

4.3 实验二：消融实验

4.3.1 实验设置

为了量化流式 ASR 与 LLM 增量预填充两项机制的相对贡献，本文挑选了 Long 及以上长度的各 50 个样本开展消融实验。按照前面章节描述的分组，本文选择了 Long (15–30s)、Very Long (30–60s) 与 Extra Long ($\geq 60\text{s}$) 三个分组，因为在该长度区间能比较好地体现流式改造的收益。对比配置设置分为三类：Baseline (System A, ASR 与 LLM 均为非流式串行执行)、Streaming ASR Only (仅 ASR 流式化, LLM 在等待全部文本到齐后再启动推理) 以及 Full Streaming (在 Streaming ASR Only 的基础上进一步启用 LLM 增量预填充, 即上文中的 System B)。

本实验的核心指标是 TTFT，另外还在日志里记录语音生成结束 endpoint 后的 ASR 尾部处理耗时以及 LLM 的预填充耗时，用来判断处理瓶颈。为了凸显架构差异给用户感知延迟带来的影响，本文在实验结果中统计了三种配置的 TTFT 均值，以及由此计算出的 ASR 增益与 KV Cache 的增益 (正值表示缩短 TTFT，负值表示增加耗时)，从而保证结论可被归因于目标机制本身。

表 4 核心模块消融实验结果
Table 4 Ablation Results of Core Modules

组名	平均 时长 (s)	Baseline TTFT(ms)	Streaming ASR Only TTFT(ms)	Full Streaming TTFT(ms)	ASR improvement(ms)	KV Cache improvement(ms)
Long	21.51	1698.88	1064.18	1084.17	634.70	-19.99
Very Long	42.43	3300.58	1171.02	1154.06	2129.56	16.96
Extra Long	83.75	6518.40	1228.77	1114.57	5289.63	114.20

4.3.2 贡献度量化分析

实验结果如表 4 所示，在实验平台硬件及所用模型条件下，ASR 流式化是缩短 TTFT 的主因。Streaming ASR Only 与 Baseline 相比，在 Long、Very Long 与 Extra Long 三组长度下分别减少了 634.70ms、2129.56ms 与 5289.63ms，对应降幅为 37.4%、64.5% 与 81.1%。这表明在传统串行链路中，语音结束 endpoint 后的主要耗时来自于 ASR 对整个音频的处理。而流式 ASR 把大部分计算提前到与语音生成的时间重叠，因此显著压缩了 endpoint 后的计算量。增量预填充在 LLM 方面表现出显著的长度依赖特性。在 Long 组，Full Streaming 相比 Streaming ASR Only 的 TTFT 反而增加了 19.99ms (KV 增益为 -19.99ms)，这说明当上下文还没足够长的时候，增量调用、缓存管理以及同步这些额外开销或许会超出其节省的一次性预填充计算。而在 Very Long 与 Extra Long 组，KV 增益分别为 16.96ms 与 114.20ms，且在 Extra Long 组，KV Cache 相对整体的收益占比达到约 9.3% ($114.20/1228.77$)，这表明，随着输入文本与上下文长度增长，LLM 预填充的收益愈加显著。

综合两项优化机制，Full Streaming 将 TTFT 均值控制在 1.1s (Long 1084.17ms、Very Long 1154.06ms、Extra Long 1114.57ms) 左右，相较于 Baseline 分别缩减了约 36.2%、65.0% 与 82.9%。该结果与实验总体趋势一致，流式并行能明显降低级联系统在输入时长增加时的等待时间，不过 KV Cache 预填充的边际收益还是受工程实现和硬件并发与算力的影响。消融实验表明，在实验平台的硬件条件下，运行 Whisper 模型已显吃力。而 Qwen2-7B-Instruct 的体量在 LLM 领域算是小型模

型，其推理计算量远小于高性能模型（参数量往往在数百到上千 B），因此实验平台的算力足以应对。如果换为主流 SOTA 模型，相信流式架构在 KV Cache 增量预填充上的收益将会更加明显。另外值得注意的是，三组递增长度的音频实验结果显示，只使用流式 ASR 和一次性预填充时，耗时还是逐渐递增的；而在全流式（使用了 KV Cache）的结果中，三组音频的处理时长并没有递增的关系。这表示在实验环境下，整个转录文本的平方阶复杂度已经超出了硬件并行计算能力的上限，需要花费更长的时间来计算注意力。而在使用 KV Cache 的情况下，尾部计算复杂度相对于整个文本接近线性阶，并未达到硬件的并行处理上限，因此即使长度增加，处理时长也并未增加，说明 KV Cache 增量预填充机制确实能在长语音场景下显著降低计算成本。

4.4 实验三：准确率与质量边界

4.4.1 实验说明与评估方法

延迟被大幅压短后，流式化的主要代价落在上下文截断与端点不确定性上。与 System A 可利用整段音频不同，流式 ASR 在每个输入的语音片段上只能看到整段音频局部的片段，句末边界附近因此更易出现替换、插入或省略。为获得稳定的识别结果，本文在 ASR 侧采用滑动窗口上下文管理，通过前缀（prefix）提供历史音频，通过后缀（suffix）提供后续音频，在精度与耗时之间进行折中。借助该机制，本实验对比三种配置：prefix=1, suffix=0（默认）、prefix=1, suffix=1（增加 1 段前瞻以改善句尾效果）以及 prefix=0, suffix=0（完全去除上下文以追求最小延迟），并从中、长、很长三组各抽取 50 条样本，共 150 条，用于覆盖中长语音的主要应用场景。本文采用业界标准的 WER 与 CER 作为衡量指标，分别针对英文（MultiWOZ）与中文（CrossWOZ）样本计算转录误差，并将非流式全量识别 System A 作为上界对照，比较 System B 在不同流式上下文配置下的识别质量与尾部耗时。

4.4.2 精度-延迟分析

表 5 不同上下文窗口配置下的准确率与耗时对比

Table 5 Accuracy and Processing Time Comparison Under Different Context Window Configurations

上下文配置	multiwoz	multiwoz	crosswoz	crosswoz	ASR 耗时 (ms)
	wer_mean	cer_mean	wer_mean	cer_mean	
前缀 1 段 + 后缀 1 段	0.0895	0.0196	0.0083	0.0203	1327.48
前缀 1 段 + 无后缀（默认配置）	0.0813	0.0221	0.0118	0.0292	1224.96
前缀 0 段 + 后缀 0 段	0.0619	0.0226	0.0210	0.0500	1086.16

实验中每种上下文配置均包含 150 条样本，其中 MultiWOZ 74 条、CrossWOZ 76 条。由于评测音频由原始对话文本经 TTS 合成生成，本文将 TTS 合成前的文本作为参考转写文本，并在计算 WER/CER 前对 ASR 输出与参考文本进行统一规范化处理，包括大小写、标点、空格及中英文分词方式的一致化。随后采用编辑距离对齐方法计算 WER 与 CER。在该合成语音集合上，非流式 System A 使用完整音频进行一次性转录，其 WER/CER 近似为 0，表明参考文本与合成音频在本实验设置下具有较高一致性；该结果仅作为本批合成数据上的上界对照，并不意味着模型在开放真实语音场景中不存在识别误差。为进一步减少文本合成与对齐过程引入的偏差，实验对异常指标样本进行了复核，并在结果分析中主要关注不同上下文配置下的相对差异。

表 5 展示了不同上下文窗口配置下的准确率与尾部耗时对比结果。可以看出,增加前后缀(pre1&suf1)对中文句尾稳定性的提升更加明显:中文数据集 CrossWOZ 的 WER/CER 分别由默认配置的 0.0118/0.0292 下降到了 0.0083/0.0203。由于处理片段长度增加,也导致了语音结束端点后的尾部耗时上升,使 asr_time_ms 由 1224.96ms 增加到了 1327.48ms (+102.52ms),这与后缀窗口需要等待额外音频片段并完成更多解码计算的机制预期相符。而对于英文数据集的 MultiWOZ, suffix 的增加使 CER 从 0.0221 降低到了 0.0196,不过 WER 稍微升高到了 0.0895,说明在以词为单位的评价标准下,后缀窗口对齐和分词的影响可能更敏感了。这一现象有必要通过真实录音及更大样本数据做进一步验证。当完全去除额外上下文(pre0&suf0)时,系统获得了更低的耗时(asr_time_ms 下降至 1086.16ms,相比默认耗时减少 138.80ms),中文误差在预料之内明显变大了:CrossWOZ 的 CER 上升到了 0.0500,WER 也上升到了 0.0210。该结果显示, prefix 给出的那些历史上下文在流式识别里发挥了重要的约束作用,能给音频-语言模型的局部决策带来更强纠正效果,从而抑制长句里逐步累积的误差。

由于中文是单字结构,而英文是单词组合,因此本文在中文数据集上主要考察 CER,在英文数据集上主要考察 WER。从中文数据集上看,随着上下文的扩大,CER 有明显降低;而在英文数据集上,三个配置的 WER 可以说基本一致,最小的上下文有最低的 WER 可以视为实验波动。这主要是因为中文的同音字较多,更多的上下文对于转录出准确的文字有较大帮助;而英文单词的同音词汇就少得多,因此只要发音准确,词级识别就相对稳定,增加的上下文并未显著提升准确率,因此在实际部署时,英文对话可去掉前后缀扩展,把流式架构的延迟优势压到极限。

最后从端到端延迟预算看,实验一与实验二显示 Long 及以上语音的 TTFT 已可稳定压缩至约 1.1s,而表 5 中不同上下文配置带来的 0.1–0.2s 尾部耗时已占去可观比例,因此需按场景进行取舍:对识别正确性更敏感的中文任务,可启用 pre1&suf1 以降低句尾错误;pre0&suf0 则作为极低延迟模式,或用于英文对话。考虑到对话式 LLM 在语义层面通常对噪声具有一定鲁棒性,转录文本的小误差预计对 LLM 的推理不会造成明显影响。后续可引入语义一致性与任务完成率等指标,直接量化转写误差造成的语义漂移程度。

5 总结与展望

5.1 全文总结

本文围绕级联式语音对话系统在长语音输入下端点后等待时间过长的问题,设计并实现了一套基于流水线并行的低延迟语音对话架构。该架构以流式计算和状态增量为设计主线,将传统串行链路中的 ASR 全量转录与 LLM 一次性预填充改造为可重叠执行的流式处理过程:在 ASR 侧,通过 VAD 分段、滑动窗口和前缀/后缀上下文,把偏向离线处理的 Whisper 封装为可输出稳定增量文本的流式模块;在 LLM 侧,利用 past_key_values 对新增文本片段进行增量预填充,并复用预填充阶段最后一步 logits 启动首 token 解码。需要指出的是,该策略并不改变注意力计算的复杂度阶数,而是把原本集中在语音结束端点之后的预填充计算提前到用户说话过程中,从而减少端点后的关键路径耗时。

实验结果验证了该架构的有效性。在从 MultiWOZ 与 CrossWOZ 构建的长语音数据集上,本文系统比传统非流式基线在 15 秒以上的长语音分组中具有稳定的优势,其将 TTFT 保持在了一个稳定的范围内。同时通过滑动窗口上下文维持了语音转录文本的准确性,WER/CER 处于可接受范围内。虽然在 Long 及以上分组的 TTFT 仍约为 1.1s,与人类对话中约 200ms 的对话间隙仍有一定差距,但与传统串行方式相比,其最大的优势在于长语音场景下的 TTFT 在达到一定数值后不再随着语音时长增长^[21]。未来借助更高效的推理硬件、更低延迟的 ASR 模型及进一步调度优化,继

续压缩这段端点后等待时间，即可实现任意长度语音均保持与人类对话间隙一致的响应延迟。

5.2 研究局限性

尽管本文降低了长语音场景下的端点后等待，但系统仍受级联架构约束。首先，当前原型仍遵循用户发言-系统处理-系统回复的半双工话轮模式，尚不能处理自然对话中的重叠说话与打断^[22]。其次，ASR 与 LLM 之间仍以文本作为唯一接口，语调、情绪、停顿等副语言信息会在转写过程中丢失。再次，本文测试音频主要由 TTS 合成，真实噪声、口音和麦克风条件下的鲁棒性仍需进一步验证。

5.3 未来工作

未来工作将集中在两方面。第一，抑制尾部延迟抖动：实验日志显示，部分长语音样本的峰值延迟来自末端连续分段触发，后续可通过末端合并、队列回压和自适应调度减少重复 ASR 推理。第二，优化话轮结束判定：真实对话通常依赖静音阈值判断用户是否结束发言，当模型端 TTFT 已压缩到约 1 s 后，固定静音等待会成为新的主导项。后续可引入轻量级语义结束判别模型（End-of-Utterance, EOU），结合流式 ASR 文本和上下文预测话轮是否语义完结，从而进一步降低真实交互等待。

参考文献

- [1] OpenAI. GPT-4o system card[EB/OL]. 2024. <https://arxiv.org/abs/2410.21276>.
- [2] TAN Z H, SARKAR A K, DEHAK N. rVAD: an unsupervised segment-based robust voice activity detection method[J]. *Computer Speech & Language*, 2020, 59: 1-21.
- [3] PRABHAVALKAR R, HORI T, SAINATH T N, et al. End-to-end speech recognition: a survey[J]. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024, 32: 325-351.
- [4] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//*Advances in Neural Information Processing Systems*. Red Hook: Curran Associates, 2017, 30: 5998-6008.
- [5] LI J. Recent advances in end-to-end automatic speech recognition[J]. *APSIPA Transactions on Signal and Information Processing*, 2022, 11(1). DOI: 10.1561/116.00000050.
- [6] RADFORD A, KIM J W, XU T, et al. Robust speech recognition via large-scale weak supervision[C]//*Proceedings of the 40th International Conference on Machine Learning*. PMLR, 2023, 202: 28492-28518.
- [7] ZHENG Y, CHEN Y, QIAN B, et al. A review on edge large language models: design, execution, and applications[J]. *ACM Computing Surveys*, 2025, 57(8): 209.
- [8] ZHU X, LI J, LIU Y, et al. A survey on model compression for large language models[J]. *Transactions of the Association for Computational Linguistics*, 2024, 12: 1556-1577.
- [9] TAY Y, DEHGHANI M, BAHRI D, et al. Efficient transformers: a survey[J]. *ACM Computing Surveys*, 2022, 55(6): 109.

- [10] SETHIYA N, MAURYA C K. End-to-end speech-to-text translation: a survey[J]. *Computer Speech & Language*, 2025, 90: 101751.
- [11] YIN S, FU C, ZHAO S, et al. A survey on multimodal large language models[J]. *National Science Review*, 2024, 11(12): nwae403.
- [12] MOHAMED A, LEE H Y, BORGHOLT L, et al. Self-supervised speech representation learning: a review[J]. *IEEE Journal of Selected Topics in Signal Processing*, 2022, 16(6): 1179-1210.
- [13] Silero Team. Silero VAD: pre-trained enterprise-grade voice activity detector[EB/OL]. 2024. <https://github.com/snakers4/silero-vad>.
- [14] BUDZIANOWSKI P, WEN T H, TSENG B H, et al. MultiWOZ: a large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling[C]//*Proceedings of EMNLP 2018*. Stroudsburg: Association for Computational Linguistics, 2018: 5016-5026.
- [15] ZHU Q, HUANG K, ZHANG Z, et al. CrossWOZ: a large-scale Chinese cross-domain task-oriented dialogue dataset[J]. *Transactions of the Association for Computational Linguistics*, 2020, 8: 281-295.
- [16] TAN X, CHEN J, LIU H, et al. NaturalSpeech: end-to-end text-to-speech synthesis with human-level quality[J]. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024, 46(6): 4234-4245.
- [17] DU Z, CHEN Q, ZHANG S, et al. CosyVoice: a scalable multilingual zero-shot text-to-speech synthesizer based on supervised semantic tokens[EB/OL]. 2024. <https://arxiv.org/abs/2407.05407>.
- [18] NI J, YOUNG T, PANDELEA V, et al. Recent advances in deep learning based dialogue systems: a systematic survey[J]. *Artificial Intelligence Review*, 2023, 56(4): 3055-3155.
- [19] ZHANG Z, TAKANOBU R, ZHU Q, et al. Recent advances and challenges in task-oriented dialog systems[J]. *Science China Technological Sciences*, 2020, 63(10): 2011-2027.
- [20] YANG A, YANG B, HUI B, et al. Qwen2 technical report[EB/OL]. 2024. <https://arxiv.org/abs/2407.10671>.
- [21] STIVERS T, ENFIELD N J, BROWN P, et al. Universals and cultural variation in turn-taking in conversation[J]. *Proceedings of the National Academy of Sciences*, 2009, 106(26): 10587-10592.
- [22] SKANTZE G. Turn-taking in conversational systems and human-robot interaction: a review[J]. *Computer Speech & Language*, 2021, 67: 101178.